

Vibe12 — Edge BACnet & Cloud FDD

py-bacnet-stacks-playground / vibe_code_apps_12

May 25, 2026

- 1 Vibe12 — Edge BACnet & Cloud FDD
- 2 Vibe12 — Edge BACnet & Cloud FDD
 - 2.1 What this app does
 - 2.2 Documentation map
 - 2.3 PDF bundle
 - 2.4 Repo layout (active paths)
- 3 Edge deploy (Ansible)
- 4 Edge deploy (Ansible)
 - 4.1 Two edge roles
 - 4.2 Quick start (boss Pi)
 - 4.3 Inventory
 - 4.4 AWS IoT certificate (shared across edges)
 - 4.5 Common commands
 - 4.6 Dual BACnet stacks (boss Pi)
 - 4.7 Verify on edge
- 5 BACnet commissioning
- 6 BACnet commissioning
 - 6.1 Prerequisites
 - 6.2 Phase 1 — Discover (read-only)
 - 6.3 Phase 2 — Edit CSV
 - 6.4 Phase 3 — Enable read driver
 - 6.5 Phase 4 — Cloud
 - 6.6 Rollout checklist
 - 6.7 Safety
- 7 AWS cloud & SAM deploy
- 8 AWS cloud & SAM deploy
 - 8.1 Copy-paste checklist (full path)
 - 8.2 What is SAM?
 - 8.3 Step A — Pack on bensserver (Linux)
 - 8.3.1 Option 1 — .tar.gz (recommended)
 - 8.3.2 Option 2 — .zip (Windows-friendly)
 - 8.4 Step B — Download to Windows (optional)
 - 8.5 Step C — CloudShell: clean home **before** upload
 - 8.6 Step D — Upload from Windows
 - 8.7 Step E — Extract

- 8.8 Step F — `samconfig.toml` (required every fresh extract)
- 8.9 Step G — Build and deploy
- 8.10 Step H — Verify
- 8.11 Tear down
- 8.12 Troubleshooting
- 9 BACnet wire capture
- 10 BACnet wire capture (pcap)
 - 10.1 Enable on deploy
 - 10.2 Pull pcap to your laptop
 - 10.3 Manual capture on edge
- 11 Commissioning CSV backup
- 12 Commissioning CSV backup (Git)
 - 12.1 Layout
 - 12.2 Pull from edge after commission
 - 12.3 Commit to GitHub
 - 12.4 Do not commit
- 13 Cloud architecture reference
- 14 Vibe Code App 12B — Deployed architecture (working reference)
 - 14.1 End-to-end flow
 - 14.2 AWS resources
 - 14.3 Example URLs (your deploy may differ)
 - 14.4 CloudShell deploy (copy-paste)
 - 14.4.1 Bensserver — pack
 - 14.4.2 CloudShell — before upload
 - 14.4.3 CloudShell — extract + config + deploy
 - 14.4.4 `samconfig.toml` essentials
 - 14.5 Common fixes
 - 14.6 Web-only update
 - 14.7 Pi (Ansible)
 - 14.8 Rule Lab: Test vs Go live (today)
 - 14.9 Future TODO (scaling — not implemented)
 - 14.9.1 1. Time-chunked evaluation — **implemented (v1)**
 - 14.9.2 2. Multi-sensor / multi-site model — **partial (v2)**
 - 14.9.3 3. Separate compute from serve
 - 14.9.4 4. Incremental FDD (watermark) — **partial (scheduled only)**
 - 14.9.5 5. Operational limits to respect
 - 14.10 Tear down
- 15 FDD expression rule cookbook
- 16 Expression rule cookbook (Rule Lab)
 - 16.1 How rules run
 - 16.2 Zone starting points (DS18B20 @ ~10 s)
 - 16.2.1 Shipped defaults (BRICK Zone_Air_Temperature_Sensor)
 - 16.3 Recipe 1 — Flatline (1 hour window)
 - 16.4 Recipe 2 — Rate of change (1 hour, net °F/hr)
 - 16.5 Recipe 3 — Rate of change (15 minutes)
 - 16.6 Recipe 4 — Spread / MIN-MAX (1 hour)
 - 16.7 Recipe 5 — Spread / MIN-MAX (15 minutes)
 - 16.8 Recipe 14 — Peak spread / MIN-MAX (24 hours)
 - 16.9 Recipe 6 — Out of bounds (instant, rolling avg)
 - 16.10 Recipe 7 — Flatline (N consecutive samples)

- 16.11 Recipe 8 — Out of bounds (debounced / sustained)
- 16.12 Recipe 9 — Rate instant (previous sample pair)
- 16.13 Recipe 10 — Verbose trace (debug / LLM report)
- 16.14 Recipe 11 — Batch sweep with `apply_faults` (advanced)
- 16.15 Recipe 12 — SAT-RAT spread (multi-sensor / Brick)
- 16.16 Recipe 13 — All SAT sensors flatline (by Brick class)
- 16.17 Quick workflow
- 16.18 Related

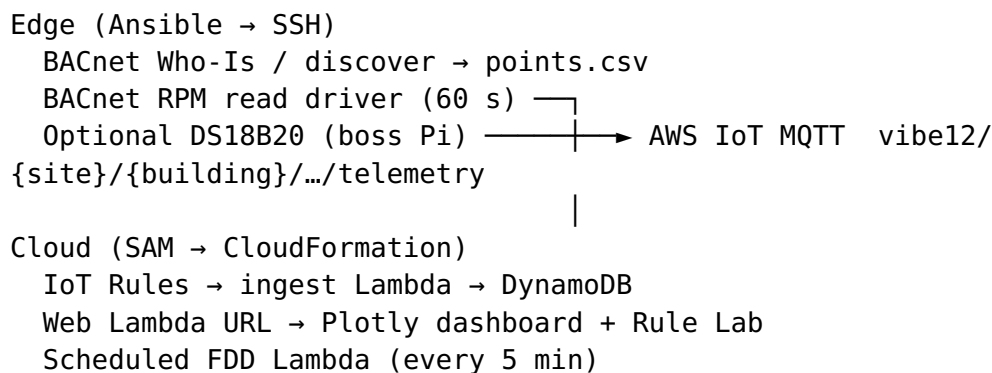
Generated May 25, 2026

1 Vibe12 — Edge BACnet & Cloud FDD

2 Vibe12 — Edge BACnet & Cloud FDD

Hands-on stack for **field BACnet gateways** (Raspberry Pi or Linux), **AWS IoT Core** telemetry, and a **serverless FDD dashboard** (DynamoDB + Lambda).

2.1 What this app does



2.2 Documentation map

Chapter	Topic
Edge deploy (Ansible)	Inventory, deploy, certs, boss Pi vs building gateway
BACnet commissioning	

Chapter	Topic
	Discover → trim CSV → enable read driver
AWS cloud & SAM deploy	tar/zip , Windows upload, CloudShell, sam deploy
Wire capture (pcap)	Post-deploy bacnet.pcap for Wireshark
Commissioning backup	fetch_commissioning.sh → Git
Cloud architecture (reference)	Deployed stack, URLs, resources
FDD rule cookbook	Rule Lab Python recipes

2.3 PDF bundle

From the repo (bensserver or dev machine):

```
cd vibe_code_apps_12
sudo apt install pandoc
./scripts/setup_docs_venv.sh
./scripts/build_docs.sh
```

Output: [pdf/vibe12-edge-fdd-guide.pdf](#) (also committed on develop for offline reading).

2.4 Repo layout (active paths)

Path	Role
ansible/	Deploy playbook, inventory, host_vars
edge_bacnet/	Discover, RPM read driver, MQTT payloads
commissioning/	Git-backed points.csv per site/building
aws_cloud_pipeline/	SAM template.yaml, Lambdas
scripts/	bacnet_tcpdump_once.sh, build_docs_pdf.py
captures/	Doc for on-edge bacnet.pcap

Historical demos live in vibe_code_apps_1 ... vibe_code_apps_11; **active development** is here in **vibe_code_apps_12**.

3 Edge deploy (Ansible)

4 Edge deploy (Ansible)

Ansible pushes the Vibe12 stack from your **build machine** (bensserver) to edge hosts over **SSH**. The Pi does **not** need a git clone — only selected files are copied.

```
bensserver: ~/py-bacnet-stacks-playground/  
            | ansible-playbook / deploy.sh
```



```
edge Pi: ~/vibe_code_apps_12/
```

Beginner walkthrough: [ansible/ANSIBLE-BEGINNER.md](#).

4.1 Two edge roles

Role	Example host	GPIO	MQTT
Building gateway	tower_a_edge	Off	BACnet read driver only
Boss Pi test bench	bacnet_pi @ 192.168.204.12	DS18B20 + BACnet MS/TP	One basicPubSub client (GPIO + BACnet)

4.2 Quick start (boss Pi)

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12/ansible  
./prepare_aws_iot_certs.sh      # once on bensserver  
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --  
ask-pass --ask-become-pass -v
```

Optional 5-minute BACnet wire capture:

```
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --  
pcap --ask-pass --ask-become-pass -v
```

4.3 Inventory

Edit [ansible/inventory.yml](#):

```
bacnet_pi:
  ansible_host: 192.168.204.12
  ansible_user: ben
```

Per-building MQTT scope: `ansible/host_vars/<host>.yaml` — `site_id`, `building_id`.

4.4 AWS IoT certificate (shared across edges)

1. Run `./prepare_aws_iot_certs.sh` → `ansible/files/aws_iot/`
2. Deploy copies PEM/key to `~/vibe_code_apps_12/aws_iot_certs/` on each host
3. IoT **policy** must allow publish to `vibe12/*` and connect as **basicPubSub** (current lab cert) or extend policy per `aws_iot_core_test/policy-vibe12-multi-client.json`

4.5 Common commands

Goal	Command
Deploy one host	<code>./deploy.sh --limit HOST -v</code>
Enable BACnet polling	<code>-e enable_bacnet_read_driver=true</code>
Backup commissioned CSV	<code>./fetch_commissioning.sh --limit HOST -v</code>
Deploy + 5 min pcap	<code>--pcap</code>
Verify only	<code>./deploy.sh --verify</code>

4.6 Dual BACnet stacks (boss Pi)

- **PiTemp** — `bacnet-ds18b20.service`, UDP **47808**
- **Vibe12Edge** — `vibe12-bacnet-read.service`, UDP **47809**

Both can run together; MQTT publishes from the read driver only.

4.7 Verify on edge

```
systemctl is-active vibe12-bacnet-read
journalctl -u vibe12-bacnet-read -n 20 --no-pager
```

MQTT test client (AWS console): `vibe12/demo/bens-office/#`

5 BACnet commissioning

6 BACnet commissioning

Commission BACnet points on the edge gateway, trim the CSV, enable the read driver, then confirm MQTT on AWS IoT.

6.1 Prerequisites

- Validated BACnet bind (--address IP/prefix or bacnet_edge_bind_address in host_vars)
- AWS IoT cert on bensserver: ansible/prepare_aws_iot_certs.sh
- site_id and building_id in **host_vars** per building
- Lab cert: MQTT client **basicPubSub**, publish vibe12/*

6.2 Phase 1 — Discover (read-only)

On the edge:

```
cd ~/vibe_code_apps_12
sudo systemctl start vibe12-bacnet-discover
journalctl -u vibe12-bacnet-discover -n 80 --no-pager
```

Output: points_discovered.csv.

MS/TP via BASRT-B (boss Pi example in host_vars/bacnet_pi.yml):

```
bacnet_route_aware: true
bacnet_router_ip: 192.168.204.200
bacnet_mstp_net: 2000
bacnet_discover_range_low: 5007
bacnet_discover_range_high: 5007
```

6.3 Phase 2 — Edit CSV

1. Open points_discovered.csv
2. Delete unwanted rows; set **enabled=1** on points to poll
3. Fill **system_id**, **brick_class**, **brick_tag**
4. Save as **points.csv**

Git backup from bensserver:

```
cd vibe_code_apps_12/ansible
./fetch_commissioning.sh --limit bacnet_pi -v
```

```
git add ../commissioning/  
git commit -m "Commission BACnet points for demo/bens-office"
```

6.4 Phase 3 — Enable read driver

```
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true -v
```

Verify:

```
ssh ben@192.168.204.12 'journalctl -u vibel2-bacnet-read -n 15 --  
no-pager'
```

Expect: published 6 samples on boss Pi (4 BACnet + 2 GPIO) every 60 s.

MQTT topic pattern:

vibel2/{site_id}/{building_id}/{system_id}/{point_id}/telemetry

6.5 Phase 4 — Cloud

Deploy SAM stack — see [AWS cloud & SAM deploy](#).

Smoke:

```
curl -sS "${URL}/api/buildings"  
curl -sS "${URL}/api/points/demo/bens-office"
```

6.6 Rollout checklist

☐

Discovery CSV exported

☐

points.csv trimmed; Brick tags filled

☐

Read driver publishing on vibel2/.../telemetry

☐

Cloud ingest shows points in dashboard

☐

One FDD rule tested in Rule Lab

6.7 Safety

- Read-only BACnet — no writes until production sign-off
- Never commit PEM/private keys to Git

7 AWS cloud & SAM deploy

8 AWS cloud & SAM deploy

Deploy the **vibe12cloud** stack (IoT rules → DynamoDB → dashboard Lambdas) from **AWS CloudShell** in **us-east-2**.

Upload target: AWS Console → **CloudShell** → **Actions** → **Upload file** (not the IoT Core console). IoT receives data from the Pi after the stack is deployed.

Prerequisites: Pi (or edge) publishing to AWS IoT; `aws sts get-caller-identity` works in CloudShell.

8.1 Copy-paste checklist (full path)

Step	Where	What
A	bensserver	<code>tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz ...</code> (below)
B	Windows	Download <code>.tar.gz</code> or <code>.zip</code> from bensserver (SCP / VS Code)
C	CloudShell	<code>rm -f old archive + rm -rf ~/aws_cloud_pipeline</code>
D	CloudShell UI	Actions → Upload file
E	CloudShell	<code>tar -xzf ...</code> or <code>unzip</code>
F	CloudShell	<code>cp samconfig.toml.example samconfig.toml</code>
G	CloudShell	<code>sam build → sam deploy --force-upload</code>
H	CloudShell	<code>curl dashboard /api/health</code>

Same content also lives in `aws_cloud_pipeline/README.md` (longer notes) and `DEPLOYED.md` (URLs after deploy).

8.2 What is SAM?

SAM packages `aws_cloud_pipeline/template.yaml` and runs **sam deploy** (CloudFormation). Stack name: **vibe12cloud**.

8.3 Step A — Pack on bensserver (Linux)

8.3.1 Option 1 — .tar.gz (recommended)

```
cd ~/py-bacnet-stacks-playground
tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz \
  -C vibe_code_apps_12 aws_cloud_pipeline
ls -lh ~/vibe12-aws-cloud-pipeline.tar.gz
```

Expect **~100 KB - 2 MB**. If the file is ~20 bytes, the pack failed.

With unit tests (optional):

```
tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz \
  -C vibe_code_apps_12 aws_cloud_pipeline tests
```

8.3.2 Option 2 — .zip (Windows-friendly)

On bensserver:

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
zip -r ~/vibe12-aws-cloud-pipeline.zip aws_cloud_pipeline
ls -lh ~/vibe12-aws-cloud-pipeline.zip
```

On Windows (PowerShell), from a folder that contains `aws_cloud_pipeline`:

```
Compress-Archive -Path aws_cloud_pipeline -DestinationPath vibe12-aws-cloud-pipeline.zip
```

Upload **either** .tar.gz or .zip to CloudShell (see below). For **zip**, extract with `unzip` instead of `tar`.

8.4 Step B — Download to Windows (optional)

Copy from bensserver to your PC (SCP, SFTP, or VS Code remote). You will upload this file through the **AWS Console**, not by pasting into the shell.

8.5 Step C — CloudShell: clean home before upload

Region: **us-east-2** → open **CloudShell**.

```
rm -f ~/vibel2-aws-cloud-pipeline.tar.gz ~/vibel2-aws-cloud-  
pipeline.zip  
rm -rf ~/aws_cloud_pipeline ~/vibe_code_apps_12  
ls ~
```

CloudShell does **not** overwrite an existing upload with the same filename — delete first.

8.6 Step D — Upload from Windows

1. AWS Console → **CloudShell** (us-east-2)
2. **Actions** → **Upload file**
3. Select vibel2-aws-cloud-pipeline.tar.gz (or .zip)

Confirm size:

```
ls -lh ~/vibel2-aws-cloud-pipeline.tar.gz  
test -f ~/vibel2-aws-cloud-pipeline.tar.gz && \  
test $(stat -c%s ~/vibel2-aws-cloud-pipeline.tar.gz) -gt 50000  
&& \  
echo "OK: tarball ready" || echo "ABORT: re-upload"
```

For zip:

```
ls -lh ~/vibel2-aws-cloud-pipeline.zip
```

8.7 Step E — Extract

Tar:

```
cd ~  
tar -xzf ~/vibel2-aws-cloud-pipeline.tar.gz
```

```
cd ~/aws_cloud_pipeline
ls
```

You should see `template.yaml`, `samconfig.toml.example`, `ingest_lambda/`, `web_lambda/`, `fdd_lambda/`.

Zip:

```
cd ~
unzip -o ~/vibe12-aws-cloud-pipeline.zip -d ~
cd ~/aws_cloud_pipeline
```

8.8 Step F — `samconfig.toml` (required every fresh extract)

```
cd ~/aws_cloud_pipeline
cp samconfig.toml.example samconfig.toml
```

Do **not** use `sam deploy --guided` again after `samconfig.toml` exists (can corrupt IoT topic parameters).

First-time only: `sam deploy --guided` and answer `sdk/test/python` for legacy topic if prompted.

8.9 Step G — Build and deploy

```
cd ~/aws_cloud_pipeline
rm -rf .aws-sam
sam build --no-cached
sam validate --lint
sam deploy --force-upload
```

Hierarchical BACnet ingest rule is included: `vibe12/+/+/+/+/telemetry`.

8.10 Step H — Verify

```
export AWS_REGION=us-east-2
export STACK=vibe12cloud
URL=$(aws cloudformation describe-stacks --stack-name "$STACK" --
region "$AWS_REGION" \
--query "Stacks[0].Outputs[?
OutputKey=='DashboardUrl'].OutputValue" --output text | tr -d
'\n\r')
URL="${URL%/*}"
```

```
echo "Dashboard: $URL"
curl -sS "${URL}/api/health" | python3 -m json.tool
```

IoT test client: subscribe to vibel2/demo/bens-office/# and sdk/test/python (legacy).

8.11 Tear down

```
sam delete --stack-name vibel2cloud
```

8.12 Troubleshooting

Problem	Fix
Upload seems ignored	<code>rm -f ~/vibel2-aws-cloud-pipeline.tar.gz</code> then re-upload
Missing option '--stack-name'	<code>cp samconfig.toml.example samconfig.toml</code>
Tarball missing in CloudShell	Confirm <code>ls -lh</code> shows > 50 KB before extract
Stack name	Use vibel2cloud only (no hyphens in some resource names)

More detail: [DEPLOYED.md](#) (stack outputs, example URLs).

9 BACnet wire capture

10 BACnet wire capture (pcap)

Validate RPM polling and MS/TP routing with a short **tcpdump** after each deploy.

10.1 Enable on deploy

```
cd vibe_code_apps_12/ansible
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --
pcap
```

Variable	Default
deploy_pcap_seconds	300 (5 minutes)
deploy_pcap_wait_seconds	30
Output file	~/vibe_code_apps_12/captures/ bacnet.pcap

Boss Pi filter: UDP **47808** and **47809** (PiTemp + Vibe12Edge).

Ansible waits until vibe12-bacnet-read is **active**, then starts capture in the background. Each deploy **overwrites** the same filename.

10.2 Pull pcap to your laptop

After capture finishes:

```
scp ben@192.168.204.12:~/vibe_code_apps_12/captures/bacnet.pcap .
wireshark bacnet.pcap
```

Wireshark display filter: bacnet or udp.port == 47808.

10.3 Manual capture on edge

```
~/vibe_code_apps_12/scripts/bacnet_tcpdump_once.sh \  
~/vibe_code_apps_12/captures/bacnet.pcap \  
"udp port 47808 or udp port 47809" \  
300
```

Same pattern as vibe_code_apps_14/captures.

11 Commissioning CSV backup

12 Commissioning CSV backup (Git)

Device configuration for each building lives in `commissioning/{site_id}/{building_id}/` in the repo — not on the Pi alone.

12.1 Layout

```
commissioning/  
  demo/  
    bens-office/  
      points.csv           # commissioned scrape list  
      points_discovered.csv # optional full discover export  
      host.yml            # metadata sidecar (Ansible fetch)
```

12.2 Pull from edge after commission

```
cd vibe_code_apps_12/ansible  
./fetch_commissioning.sh --limit bacnet_pi -v
```

12.3 Commit to GitHub

```
cd ~/py-bacnet-stacks-playground  
git add vibe_code_apps_12/commissioning/  
git commit -m "Commission BACnet points for demo/bens-office"  
git push origin develop
```

Next deploy pushes `commissioning/.../points.csv` back to the edge automatically when the file exists in the repo.

12.4 Do not commit

- `aws_iot_certs/` or `ansible/files/aws_iot/*.key`
- `captures/*.pcap`

13 Cloud architecture reference

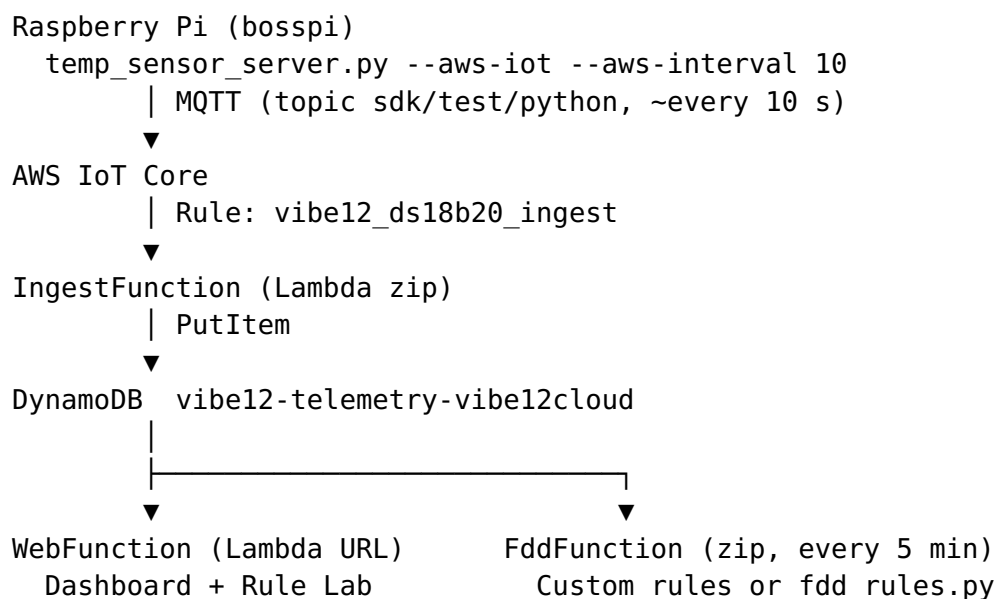
14 Vibe Code App 12B — Deployed architecture (working reference)

Working stack: **vibe12cloud** in **us-east-2** — Pi DS18B20 → IoT → DynamoDB → dashboard + Rule Lab.

Rule recipes: **EXPRESSION_RULE_COOKBOOK.md**

Full deploy steps: **README.md**

14.1 End-to-end flow



14.2 AWS resources

Resource	What it is
DynamoDB vibe12-telemetry-vibe12cloud	Temperature history (7-day TTL)
S3 aws-sam-cli-managed-...	SAM deploy artifacts only MQTT → ingest Lambda

Resource	What it is
IoT Rule vibe12_ds18b20_ingest	
DashboardUrl	Public HTTPS (Lambda Function URL)

14.3 Example URLs (your deploy may differ)

- **Dashboard:** `https://mlmdwoonvb5bgltfy7dgiqv7mq0amllu.lambda-url.us-east-2.on.aws/`
- **JSON:** same host + `/api/readings?hours=6`
- **Health:** same host + `/api/health`

14.4 CloudShell deploy (copy-paste)

14.4.1 Bensserver — pack

```
tar -czf /home/ben/vibe12-aws-cloud-pipeline.tar.gz \
-C /home/ben/py-bacnet-stacks-playground/vibe_code_apps_12
aws_cloud_pipeline
ls -lh /home/ben/vibe12-aws-cloud-pipeline.tar.gz
```

14.4.2 CloudShell — before upload

```
rm -f ~/vibe12-aws-cloud-pipeline.tar.gz ~/vibe12-aws-cloud-
pipeline.zip
rm -rf ~/aws_cloud_pipeline ~/vibe_code_apps_12
```

Actions → **Upload file** → `vibe12-aws-cloud-pipeline.tar.gz`

14.4.3 CloudShell — extract + config + deploy

```
cd ~
tar -xzf ~/vibe12-aws-cloud-pipeline.tar.gz
cd ~/aws_cloud_pipeline

# Required after every fresh extract (fixes "Missing option '--
stack-name'):
cp samconfig.toml.example samconfig.toml

rm -rf .aws-sam
sam build --no-cached
```

```
sam validate --lint
sam deploy --force-upload
```

Do **not** run `sam deploy --guided` after `samconfig.toml` is correct (can save `IotTopic=y`).

14.4.4 samconfig.toml essentials

```
stack_name = "vibe12cloud"
region = "us-east-2"
resolve_s3 = true
resolve_image_repos = true
```

14.5 Common fixes

Issue	Fix
Missing option '--stack-name'	<code>cp samconfig.toml.example samconfig.toml</code>
Upload ignored	<code>rm -f ~/vibe12-aws-cloud-pipeline.tar.gz</code> then re-upload
AwsIotEventsSqlVersion	Use AwsIotSqlVersion in template
Stack name hyphen	vibe12cloud only <code>rm -rf ~/</code>
Stale code	<code>aws_cloud_pipeline</code> + full re-extract + <code>sam build --no-cached</code>
Chart zoom resets every 1 min	Deploy latest dashboard JS — Pause auto-refresh while zoomed (toolbar); Reset zoom to refit
502 / Internal S... not JSON on /api/readings	Response too large or Lambda timeout on 7 d + all rules. Deploy fix: long windows use chunked chart eval (same 6 h batches as go-live) when hours>48 or samples>8000; try 24 h if on old deploy; check CloudWatch WebFunction
Debug on dashboard	After deploy: Backend logs panel shows srv:

Issue	Fix
	lines from debug.server_log; errors show stage= and hint

14.6 Web-only update

```
cd ~/aws_cloud_pipeline
cp samconfig.toml.example samconfig.toml
sam build WebFunction
sam deploy --force-upload
```

14.7 Pi (Ansible)

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12/ansible
./deploy.sh --ask-pass --ask-become-pass -v
```

group_vars/pi_bcn.yml: aws_iot_publish_interval: 10

14.8 Rule Lab: Test vs Go live (today)

Step	Test rule (hours dropdown)	Go live (up to 7 d)
Read telemetry	One DynamoDB query for that window	One query for full backfill (cap ~62k points)
Run rules	evaluate() per row; optional (True, window_rows) or apply_faults()	Same sweep engine (chunked on go-live)
FDD DB writes	None	Rules (ts_ms=-2) + summary (ts_ms=0) only
Dashboard fault lanes	N/A	Recomputed on each /api/readings refresh (not replayed from Go live)

Go live runs **chunked AFDD — hard-coded: 6 h batches, max 7 d (168 h)** (not the Rule Lab test-window dropdown). Each batch loads only that interval from DynamoDB, evaluates rules, merges **flag counts**, then discards rows. State at `ts_ms=-3`; summary at `ts_ms=0`.

Scheduled **FddFunction** (every 5 min) does **incremental** catch-up from the watermark when rules unchanged; full chunked backfill when rules change or first run.

Fine for: one Pi, ~10 s MQTT, a few rules, teaching / demo.

Pressure points at scale: dashboard `/api/readings` still re-evaluates rules for chart lanes (downsampled); one `device_id` per stack.

Retroactive flatline / lookback windows: `evaluate()` can return `(True, window_rows)` so the engine flags the whole hour, not just the detection row. Or define `apply_faults(rows, cfg) -> list[bool]`. See **Recipe 5** in `EXPRESSION_RULE_COOKBOOK.md`.

14.9 Future TODO (scaling — not implemented)

Notes for multi-sensor / high-volume sites. Current tutorial stack stays simple on purpose.

14.9.1 1. Time-chunked evaluation — implemented (v1)

- Go live + scheduled FDD: **6 h chunks** (`FDD_CHUNK_HOURS`), overlap for rolling avg, merge counts.
- `afdd_state` at `ts_ms=-3`; summary at `ts_ms=0` includes `chunk_log` (last 40 chunks).

14.9.2 2. Multi-sensor / multi-site model — partial (v2)

- BACnet edge: `edge_bacnet/ discover` → CSV → RPM read driver → MQTT `vibe12/{site}/{building}/{system}/{point}/telemetry`
- Ingest: legacy `sdk/test/python` + hierarchical BACnet rule; `series_id` as `device_id` partition key
- DynamoDB GSI `BuildingScopeIndex` (`building_scope` + `scope_sort`)
- APIs: `/api/buildings`, `/api/points/{site}/{building}`, `/api/series`, `/api/brick/{site}/{building}`
- Dashboard multi-series overlay + Rule Lab series context (see `BACNET_COMMISSIONING.md`)

- Still TODO: FDD fan-out per building, pre-aggregated serve path at very high point counts

14.9.3 3. Separate compute from serve

- **Batch job** (scheduled or on-demand): runs rules, writes summary + compact artifacts.
- **Read API / dashboard**: serves pre-aggregated results instead of full re-sweep on every browser refresh.

14.9.4 4. Incremental FDD (watermark) — partial (scheduled only)

- **FddFunction** resumes from watermark_ms when rules_revision matches.
- **Go live** always full chunked backfill for the requested hours (resets counts).
- Future: carry debounce state across chunks in the engine (not rule-local lists).

14.9.5 5. Operational limits to respect

Limit	Today
READINGS_LIMIT	~62k samples per query/eval pass
WebFunction timeout	120 s
FDD status row	Summary only (counts, badge, eval_log) — not per-sample flag arrays
Ingest	One put_item per MQTT sample (real time)

14.10 Tear down

```
sam delete --stack-name vibe12cloud
```

15 FDD expression rule cookbook

16 Expression rule cookbook (Rule Lab)

Copy a recipe into **Rule Lab** → **Python editor**, set **Parameters (cfg)** from the table, click **Test rule** (6 h preview), then **Save draft** or **Write to database**.

16.1 How rules run

Topic	Detail
Entry point	evaluate(row, cfg, prev_row=None, rows=None) — required
Row fields	row, ts_ms, ts, temp, temp_raw, temp_rolling_avg, degF, degC, degF_rolling_avg, sample_period_ms, rolling_avg_minutes, samples_in_avg
Config	Use cfg_threshold(cfg, "key") — values follow Rule unit (°F or °C). Add keys via + Parameter or Add preset...
Helpers	temp_unit_symbol(cfg), math, datetime, optional numpy as np
Instant flag	return True — flags this row only
Retroactive lane	return True, window_rows — paints every row in window_rows on the chart (recommended for lookback rules)
Batch mode	Optional apply_faults(rows, cfg) -> list[bool] — see Recipe 10

Tip: At ~10 s MQTT, 6 samples ≈ 1 minute, 360 samples ≈ 1 hour. Use FILL_RATIO = 0.95 so the window is nearly full before evaluating.

16.2 Zone starting points (DS18B20 @ ~10 s)

Recipe	Config key(s)	Start values (°F / °C)
1 — Flatline 1 h	flatline_tolerance	0.10 / 0.03
2 — Rate 1 h	max_temp_per_hour	5.0 / 3.0
3 — Rate 15 m	max_temp_per_15min	2.0 / 1.1
4 — Spread 1 h	max_spread	4.0 / 2.2
5 — Spread 15 m	max_spread_15min	2.5 / 1.4
6 — Out of bounds	bounds_low, bounds_high	65, 80 / 18, 27
7 — Flatline N samples	flatline_tolerance, flatline_window	0.05 / 0.03, 18
8 — OOB debounced	bounds_low, bounds_high, rolling_window	65, 80, 6
9 — Rate instant (pair)	max_temp_per_minute	2.0 / 1.1
14 — Peak spread 24 h	max_spread_24h	12.0 / 6.7

16.2.1 Shipped defaults (BRICK Zone_Air_Temperature_Sensor)

Rule Lab loads four enabled rules from `rules_defaults.py` when no DynamoDB draft exists. Each includes `brick_scope.point_classes: ["Zone_Air_Temperature_Sensor"]` so **Test across BRICK targets** and scheduled FDD evaluate every matching zone sensor.

Default rule ID	Cookbook	What it detects
brick_zone_oob	Recipe 6	Comfort band violation (rolling avg)
brick_zone_flatline_1h	Recipe 1	Stuck sensor (1 h min-max spread)
brick_zone_swing_15m	Recipe 5	

Default rule ID	Cookbook	What it detects
brick_zone_peak_swing_24h	Recipe 14	Short hunting / fast swing (15 m min-max) Excessive daily peak-valley (24 h min-max)

Note: Recipes 2–3 measure **net endpoint rate** (first → last). Recipes 4–5 and 14 measure **peak swing** (max – min). Use spread recipes when a spike that returns to baseline should still fault.

Test windows: Use ≥ 2 h for 1 h / 15 m lookback rules; use ≥ 24 h (or max history) when testing brick_zone_peak_swing_24h.

16.3 Recipe 1 — Flatline (1 hour window)

Sensor stuck: max – min over the last hour is below tolerance. Paints the full hour when detected.

Config: flatline_tolerance = **0.10** (°F) or **0.03** (°C)

Test tip: Use **Test window ≥ 2 h** (default). print() runs only when a fault fires; enable **Verbose prints (window trace)** to see sample 1 h spreads without changing code.

```
ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95
```

```
def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]
```

```
def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False
```

```
    window_rows = get_last_1_hour(row, rows)
    if len(window_rows) < 2:
        return False
```

```
    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False
```



```

sym = temp_unit_symbol(cfg)
vals = [r["temp"] for r in window_rows]
spread = max(vals) - min(vals)
tol = cfg_threshold(cfg, "flatline_tolerance")

if spread < tol:
    print(
        f"row={row['row']} ts={row['ts']} "
        f"FLATLINE 1h spread={spread:.3f} {sym} < tol={tol:.3f}"
    )
    return True, window_rows

return False

```

16.4 Recipe 2 — Rate of change (1 hour, net °F/hr)

Net temperature change over a full hour, divided by elapsed hours.

Config: max_temp_per_hour = **5.0** (°F/hr) or **3.0** (°C/hr)

```

ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95

```

```

def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_1_hour(row, rows)
    if len(window_rows) < 2:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False

    dt_hr = (window_rows[-1]["ts_ms"] - window_rows[0]
["ts_ms"]) / 3600000.0
    if dt_hr <= 0:
        return False

```

```

sym = temp_unit_symbol(cfg)
v0 = window_rows[0]["temp"]
v1 = window_rows[-1]["temp"]
rate_hr = abs(v1 - v0) / dt_hr
lim = cfg_threshold(cfg, "max_temp_per_hour")

print(
    f"row={row['row']} ts={row['ts']} "
    f"rate={rate_hr:.2f} {sym}/hr ({v0:.2f} -> {v1:.2f}) "
)

if rate_hr > lim:
    print(f"RATE/Hr: painting {len(window_rows)} rows")
    return True, window_rows

return False

```

16.5 Recipe 3 — Rate of change (15 minutes)

Same as Recipe 2, but normalized to a 15-minute equivalent rate.

Config: max_temp_per_15min = **2.0** (°F) or **1.1** (°C)

```

FIFTEEN_MIN_MS = 15 * 60 * 1000
FILL_RATIO = 0.95

```

```

def get_last_15_min(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - FIFTEEN_MIN_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]

```

```

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

```

```

    window_rows = get_last_15_min(row, rows)
    if len(window_rows) < 2:
        return False

```

```

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < FIFTEEN_MIN_MS * FILL_RATIO:
        return False

```

```

    dt_min = (window_rows[-1]["ts_ms"] - window_rows[0]
["ts_ms"]) / 60000.0

```

```

if dt_min <= 0:
    return False

sym = temp_unit_symbol(cfg)
delta = abs(window_rows[-1]["temp"] - window_rows[0]["temp"])
rate_15 = delta * (15.0 / dt_min)
lim = cfg_threshold(cfg, "max_temp_per_15min")

print(
    f"row={row['row']} ts={row['ts']} "
    f"equiv {rate_15:.2f} {sym} per 15 min"
)

if rate_15 > lim:
    print(f"RATE/15m: painting {len(window_rows)} rows")
    return True, window_rows

return False

```

16.6 Recipe 4 — Spread / MIN-MAX (1 hour)

Zone swung too much: peak – valley over one hour exceeds limit (short cycling, stuck damper, etc.).

Config: max_spread = **4.0** (°F) or **2.2** (°C)

```

ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95

```

```

def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_1_hour(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False

```

```

sym = temp_unit_symbol(cfg)
vals = [r["temp"] for r in window_rows]
lo, hi = min(vals), max(vals)
spread = hi - lo
lim = cfg_threshold(cfg, "max_spread")

print(
    f"row={row['row']} ts={row['ts']} "
    f"spread={spread:.2f} {sym} (min={lo:.2f} max={hi:.2f})"
)

if spread > lim:
    print(f"SPREAD/1h: painting {len(window_rows)} rows")
    return True, window_rows

return False

```

16.7 Recipe 5 — Spread / MIN-MAX (15 minutes)

Short-window spread — catches fast hunting or sudden step changes.

Config: max_spread_15min = **2.5** (°F) or **1.4** (°C)

```

FIFTEEN_MIN_MS = 15 * 60 * 1000
FILL_RATIO = 0.95

```

```

def get_last_15_min(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - FIFTEEN_MIN_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_15_min(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < FIFTEEN_MIN_MS * FILL_RATIO:
        return False

    sym = temp_unit_symbol(cfg)
    vals = [r["temp"] for r in window_rows]

```

```

lo, hi = min(vals), max(vals)
spread = hi - lo
lim = cfg_threshold(cfg, "max_spread_15min")

print(
    f"row={row['row']} ts={row['ts']} "
    f"spread={spread:.2f} {sym} (min={lo:.2f} max={hi:.2f})"
)

if spread > lim:
    print(f"SPREAD/15m: painting {len(window_rows)} rows")
    return True, window_rows

return False

```

16.8 Recipe 14 — Peak spread / MIN-MAX (24 hours)

Daily peak-valley swing: catches hunting, short cycling, or setpoint fighting that cancels out over shorter windows. Same semantics as Recipe 4-5, but over a full day. **Shipped default:** brick_zone_peak_swing_24h.

Config: max_spread_24h = **12.0** (°F) or **6.7** (°C)

Test tip: Use **Test window ≥ 24 h** (or max dashboard history). Requires ~95% fill of the 24 h window before evaluating.

```

TWENTY_FOUR_HOUR_MS = 24 * 60 * 60 * 1000
FILL_RATIO = 0.95

```

```

def get_last_24_hours(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - TWENTY_FOUR_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_24_hours(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < TWENTY_FOUR_HOUR_MS * FILL_RATIO:
        return False

```

```

sym = temp_unit_symbol(cfg)
vals = [r["temp"] for r in window_rows]
lo, hi = min(vals), max(vals)
spread = hi - lo
lim = cfg_threshold(cfg, "max_spread_24h")

print(
    f"row={row['row']} ts={row['ts']} "
    f"peak spread 24h={spread:.2f} {sym} (min={lo:.2f}
max={hi:.2f})"
)

if spread > lim:
    print(f"PEAK/24h: painting {len(window_rows)} rows")
    return True, window_rows

return False

```

16.9 Recipe 6 — Out of bounds (instant, rolling avg)

Flags the current row when smoothed temperature is outside the band. Matches the shipped default **brick_zone_oob** rule.

Config: `bounds_low = 65`, `bounds_high = 80` (°F) — or **18 / 27** (°C)

Set **Rolling avg** in the toolbar to **1 min** (or **10 min** for slower response).

```

def evaluate(row, cfg, prev_row=None, rows=None):
    sym = temp_unit_symbol(cfg)
    low = cfg_threshold(cfg, "bounds_low")
    high = cfg_threshold(cfg, "bounds_high")

    # Prefer rolling avg when present (Rule Lab computes from
    toolbar setting)
    if "temp_rolling_avg" in row:
        v = row["temp_rolling_avg"]
        kind = "avg"
    elif "degF_rolling_avg" in row:
        v = row["degF_rolling_avg"]
        kind = "degF_avg"
    else:
        v = row["temp"]
        kind = "raw"

    if v < low or v > high:
        print(

```

```

        f"{row['ts']} 00B {kind} {v:.2f} {sym} "
        f"(band {low:.1f}–{high:.1f}, raw={row['temp']:.2f})"
    )
    return True

return False

```

16.10 Recipe 7 — Flatline (N consecutive samples)

Sample-count window (not wall-clock). Good when MQTT spacing is steady (~ 10 s $\rightarrow$ 18 samples $\approx$ 3 min).

Config: flatline_tolerance = **0.05**, flatline_window = **18**

```

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    sym = temp_unit_symbol(cfg)
    w = int(cfg.get("flatline_window", 18))
    if w < 2:
        w = 2
    if row["row"] < w - 1:
        return False

    i = row["row"]
    win = rows[i - w + 1 : i + 1]
    vals = [r["temp"] for r in win]
    tol = cfg_threshold(cfg, "flatline_tolerance")
    spread = max(vals) - min(vals)

    if spread < tol:
        print(
            f"{row['ts']} FLATLINE w={w} spread={spread:.3f}
{sym} "
            f"tol={tol:.3f}"
        )
        return True, win

    return False

```

16.11 Recipe 8 — Out of bounds (debounced / sustained)

Requires **every** sample in the last N rows to be out of band before flagging (~ 1 min at 6×10 s). Reduces flicker from noise.

Config: `bounds_low = 65`, `bounds_high = 80`, `rolling_window = 6`
Add `rolling_window` with + **Parameter** (integer, default 6).

```
def _oob(r, low, high):
    v = r.get("temp_rolling_avg", r["temp"])
    return v < low or v > high

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    sym = temp_unit_symbol(cfg)
    low = cfg_threshold(cfg, "bounds_low")
    high = cfg_threshold(cfg, "bounds_high")
    n = int(cfg.get("rolling_window", 6))
    if n < 1:
        n = 1

    if row["row"] < n - 1:
        return False

    i = row["row"]
    win = rows[i - n + 1 : i + 1]

    if all(_oob(r, low, high) for r in win):
        v = win[-1].get("temp_rolling_avg", win[-1]["temp"])
        print(
            f"{row['ts']} 00B sustained {n} samples "
            f"last={v:.2f} {sym} band {low:.1f}–{high:.1f}"
        )
        return True, win

    return False
```

16.12 Recipe 9 — Rate instant (previous sample pair)

Fast spike detector: rate between this row and the previous sample only. Default **Rate > limit (per minute)** style.

Config: max_temp_per_minute = **2.0** (°F/min) or **1.1** (°C/min)

```
def evaluate(row, cfg, prev_row=None, rows=None):
    if not prev_row:
        return False

    sym = temp_unit_symbol(cfg)
    dt_ms = row["ts_ms"] - prev_row["ts_ms"]
    if dt_ms <= 0:
        return False

    dt_min = dt_ms / 60000.0
    rate = abs(row["temp"] - prev_row["temp"]) / dt_min
    lim = cfg_threshold(cfg, "max_temp_per_minute")

    if rate > lim:
        print(
            f"{row['ts']} RATE/min {rate:.2f} {sym}/min "
            f"({prev_row['temp']:.2f} -> {row['temp']:.2f})"
        )
        return True

    return False
```

16.13 Recipe 10 — Verbose trace (debug / LLM report)

Prints every row when **Verbose prints** is checked in Rule Lab.
Use to learn row fields or paste console into an LLM.

Config: none required

```
def evaluate(row, cfg, prev_row=None, rows=None):
    sym = temp_unit_symbol(cfg)
    avg = row.get("temp_rolling_avg", row["temp"])
    print(
        f"#{row['row']:4d} {row['ts']} "
        f"raw={row['temp']:.2f} avg={avg:.2f} {sym} "
        f"samples_in_avg={row.get('samples_in_avg', '?')}"
    )
    return False
```

16.14 Recipe 11 — Batch sweep with apply_faults (advanced)

Runs evaluate once per row but merges painted windows in a second pass. Useful for heavy rules or custom merge logic.

Config: same as Recipe 2 (max_temp_per_hour = 5.0)

```
ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95
```

```
def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]
```

```
def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False
```

```
    window_rows = get_last_1_hour(row, rows)
    if len(window_rows) < 2:
        return False
```

```
    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False
```

```
    dt_hr = span_ms / 3600000.0
    if dt_hr <= 0:
        return False
```

```
    v0 = window_rows[0]["temp"]
    v1 = window_rows[-1]["temp"]
    rate_hr = abs(v1 - v0) / dt_hr
    lim = cfg_threshold(cfg, "max_temp_per_hour")
```

```
    if rate_hr > lim:
        return True, window_rows
    return False
```

```
def apply_faults(rows, cfg):
    flags = [0] * len(rows)
    for row in rows:
        hit, window_rows = evaluate(row, cfg, rows=rows)
        if hit:
            for w in window_rows:
```

```

        flags[w["row"]] = 1
    return flags

```

16.15 Recipe 12 — SAT-RAT spread (multi-sensor / Brick)

Cross-sensor mechanical rule using series context. Map aliases in rule **config**:

```

{
    "max_spread": 25.0,
    "series_aliases": {
        "SAT": "acme#tower-a#ahu-1#3456788-analog-input-2",
        "RAT": "acme#tower-a#ahu-1#3456788-analog-input-4"
    }
}

```

Or use Brick tags if series IDs match alias keys after graph import.

```

def evaluate(row, cfg, prev_row=None, rows=None, series=None):
    if not series:
        return False
    sat = series.get("SAT", {}).get("current")
    rat = series.get("RAT", {}).get("current")
    if sat is None or rat is None:
        return False
    spread = abs(float(sat) - float(rat))
    if spread > cfg["max_spread"]:
        print(f"{row['ts']} SAT-RAT spread={spread:.1f}")
    SAT={sat} RAT={rat}")
    return True
    return False

```

16.16 Recipe 13 — All SAT sensors flatline (by Brick class)

Use **Test rule** with building scope after multi-series data is loaded, or query `/api/series/by-tag?`
`brick_class=Supply_Air_Temperature_Sensor.`

```

def evaluate(row, cfg, prev_row=None, rows=None, series=None):
    # Primary row is first series in scope; use series["SAT"]
    ["values"] for window
    if not series or "SAT" not in series:
        return False
    vals = [v for v in series["SAT"]["values"] if v is not None]
    [-18:]

```

```
    if len(vals) < 18:
        return False
    tol = cfg.get("flatline_tolerance", 0.05)
    if max(vals) - min(vals) < tol:
        print(f"{row['ts']} SAT flatline spread={max(vals)-
min(vals):.3f}")
        return True
    return False
```

16.17 Quick workflow

1. + **Add rule** or pick an existing rule in the dropdown.
2. Paste recipe code; add cfg keys from the table (**Add preset...**).
3. Set **Rule unit** to match your thresholds (°F vs °C).
4. **Test rule** — console shows print output; chart preview uses test window only.
5. **Copy report for LLM** — full rule + sweep log for debugging.
6. **Save draft** — rules only in DynamoDB (ts_ms=-2).
7. **Write to database** — rules + 7 d FDD backfill + status row.

16.18 Related

- Default shipped rules: web_lambda/rules_defaults.py
- Retroactive painting tests: tests/test_retroactive_faults.py
- Synthetic faults on Pi: fault_demo_schedule.py +
temp_sensor_server --fault-demo